



Data Analytics mit SQL & Python

VON DATEN ZUR ENTSCHEIDUNG

DR. CHIAO-YA CHANG
CHIAOYA.CHANG.TW@GMAIL.COM

Tag 2

Erweiterte SQL-Abfragen & Datenmodellierung



Ablauf

Tag 2 – Erweiterte SQL-Abfragen & Datenmodellierung		
09:00–09:30	Wiederholung	Übung: SELECT, FROM, WHERE, ORDER BY & COUNT, SUM, AVG, MAX
09:30–10:30	GROUP BY, HAVING & JOINS	• GROUP BY & HAVING • JOINS & STAR SCHEMA
10:30–11:30	Subqueries & Datenmodellierung	• Subqueries in WHERE / FROM • Primär- & Fremdschlüssel • Relationalen Modellen (Star Schema)
11:30–12:10	 Mittagspause	—
12:10–14:00	Erweiterte SQL-Konzepte & Übung	• Common Table Expressions (CTE) • Window Functions (z. B. Ranking, laufende Summen)
14:00–14:20	 Pause	—
14:20–15:40	Praktische Übung	Mini-Projekt & Präsentation
15:40–16:00	Ergebnisse & Diskussion	Präsentation, Reflexion, Q&A





Tag 2 09:00–09:30

Wiederholung & Einstieg

- Kurze Wiederholung von:
SELECT, WHERE, ORDER BY
& COUNT, SUM, AVG, MAX
- Kleine Quizfragen oder
interaktive Übungen

Wiederholung & Einstieg

30-minütige Übung mit Fitness-Daten

Die Teilnehmenden sollten jedoch:

- **Google Colab öffnen** und die Datei „Day_2_Data_Analytics_SQL_Python_Exercises.ipynb“ laden
- **Teil A zur Wiederholung von Tag 1 bearbeiten**

Ziel:

- Aktivierung des Vorwissens
- Einstieg in die Arbeit mit realen Daten





Tag 2 09:30–10:30

GROUP BY, HAVING & JOINS

- GROUP BY & HAVING
- JOINS & STAR SCHEMA



Was Wir heute lernen?

- 1 WHERE und HAVING unterscheiden
- 2 Daten mit JOINS verbinden (INNER, LEFT)
- 3 GROUP BY für Geschäftskennzahlen nutzen
- 4 UNION verwenden, um ähnliche Tabellen zusammenzuführen
- 5 Unterabfragen verstehen und schreiben
- 6 CTEs (Common Table Expressions) verwenden
- 7 Window Functions für Rankings verwenden

SQL Grundlagen

SQL Abfragen

1 **SELECT, FROM, WHERE (HAVING), ORDER BY** Die 4 Grundbausteine

2 **COUNT, SUM, AVG, MAX** Schnelle Berechnungen

3 **Daten verknüpfen** Zusammenführung mehrerer Tabellen

4 **CTEs & Window Functions** Fortgeschrittene Analysen (z. B. Ranking, laufende Summen)



Keine Vorkenntnisse nötig!

Wir fangen gemeinsam bei Null an und arbeiten
Schritt fuer Schritt.

Werkzeug: **Google Colab** -- kein Download, direkt im Browser, kostenlos

WHERE vs HAVING

 Welche Kunden kaufen viel?

WHERE

Filtert ZEILEN **vor** Gruppierung

```
SELECT *  
FROM shop_orders  
WHERE quantity > 8;
```

HAVING

Filtert GRUPPEN **nach** Gruppierung

```
SELECT customer_id, SUM(quantity) AS  
total_quantity  
FROM shop_orders  
GROUP BY customer_id  
HAVING SUM(quantity) > 8;
```

 Merksatz: WHERE filtert Zeilen. HAVING filtert Gruppen.

```
SELECT customer_id, SUM(quantity) AS  
total_quantity  
FROM shop_orders  
WHERE SUM(quantity) > 8  
GROUP BY customer_id;
```

X Häufiger Fehler

```
SELECT customer_id, SUM(quantity) AS  
total_quantity  
FROM shop_orders  
GROUP BY customer_id  
HAVING SUM(quantity) > 8;
```

v Richtig

Geschäftsrealität

Der Chef fragt:

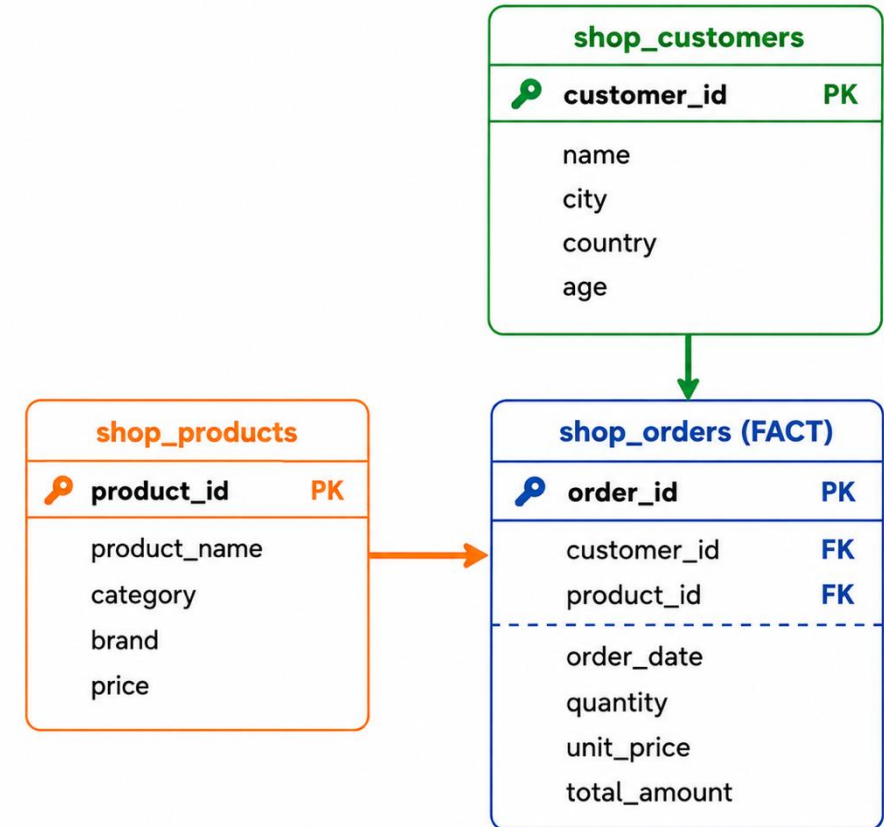
- Welche Kunden bringen den meisten Umsatz?
- Und welche Produkte verkaufen sich am besten?

Das Problem:

- Die relevanten Daten liegen in **verschiedenen Tabellen**
- Kundeninformationen → CRM
- Bestellungen → ERP-System
- Produktdaten → Produktdatenbank

Was ist ein Star Schema?

- Eine **Fact Table** in der Mitte mit Messwerten (Umsatz, Menge)
- Mehrere **Dimension Tables** aussen mit Kontext (Wer? Was? Wann?)
- Verbindung über **Foreign Keys** (customer_id, product_id)

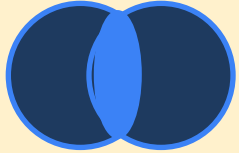


- 👉 Jede Tabelle für sich ist nicht ausreichend
- 👉 Frage: Wie verbinden wir diese Daten sinnvoll?

Genau das lösen wir jetzt mit JOINS

SQL JOINS

(Beispiel: shop_orders x shop_customers)



INNER JOIN = nur wo beide passen
nur Zeilen die in BEIDEN Tabellen passen

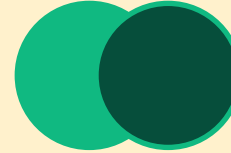
```
SELECT o.order_id, c.name, o.total_amount
FROM shop_orders o
INNER JOIN shop_customers c
ON o.customer_id = c.customer_id
```

ERGEBNIS

ORDER_ID	NAME	TOTAL_AMOUNT
1001	Anna Müller	€ 899
1003	Ben Schulz	€ 79

↳ Order 1002 hat keine customer_id → wird ausgeschlossen

- ✓ Gut für: echte Verkaufsanalyse
- ⚠ Risiko: Kunden ohne Bestellung verschwinden.



LEFT JOIN = alles links
alle Orders, auch ohne passenden Kunden

```
SELECT o.order_id, c.name, o.total_amount
FROM shop_orders o
LEFT JOIN shop_customers c
ON o.customer_id = c.customer_id
```

ERGEBNIS

ORDER_ID	NAME	TOTAL_AMOUNT
1001	Anna Müller	€ 899
1002	NULL	€ 29
1003	Ben Schulz	€ 79

↳ Order 1002 bleibt erhalten, Name = NULL

- ✓ Gut für: Kundenanalyse, inaktive Kunden finden
- ⚠ Risiko: NULL-Werte müssen erklärt werden.

SQL JOINS (optional)

(Beispiel: shop_orders x shop_customers)



RIGHT JOIN = alles rechts
alle Kunden, auch ohne Bestellung

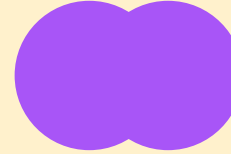
```
SELECT o.order_id, c.name, o.total_amount
FROM shop_orders o
RIGHT JOIN shop_customers c
ON o.customer_id = c.customer_id
```

ERGEBNIS

ORDER_ID	NAME	TOTAL_AMOUNT
1001	Anna Müller	€ 899
1003	Ben Schulz	€ 79
NULL	Clara Benz	NULL

↳ Clara hat noch nie bestellt → order = NULL

- ✅ Für Anfänger weniger wichtig.
- ⚠️ Eher für Datenqualität: Bestellungen ohne Kunden?



FULL OUTER JOIN = niemanden zuruecklassen
alle Zeilen aus beiden Tabellen

```
SELECT o.order_id, c.name, o.total_amount
FROM shop_orders o
FULL OUTER JOIN shop_customers c
ON o.customer_id = c.customer_id
```

ERGEBNIS

↳ Alle Zeilen erhalten — fehlende Werte = NULL

ORDER_ID	NAME	TOTAL_AMOUNT
1001	Anna Müller	€ 899
1002	NULL	€ 29
1003	Ben Schulz	€ 79
NULL	Clara Benz	NULL

- ✅ Gut für: Datenqualitätsprüfung
- ⚠️ Risiko: In SQLite nicht direkt unterstützt.



Tag 2 10:30–11:30

Subqueries & Datenmodellierung

- Subqueries in WHERE / FROM
- Primär- & Fremdschlüssel
- Relationalen Modellen (Star Schema)

Subquery

 Eine Subquery ist eine **SELECT-Abfrage INNERHALB** einer anderen Abfrage.

Analogie: Frage in einer Frage

Normale Frage:

"Zeige mir alle Kunden aus Berlin."

Subquery-Frage:

"Zeige mir alle Kunden, die mehr ausgegeben haben als der Durchschnitt (den ich erst noch berechnen muss)."

Sie berechnet zuerst ein Zwischenergebnis, das die äußere Abfrage dann verwenden kann.

Aufbau in SQL:

```
SELECT name, total_amount
FROM shop_orders
WHERE total_amount >
      (SELECT AVG(total_amount)
       FROM shop_orders);
```

Äußere Abfrage

Innere Abfrage (Subquery)

Die Subquery wird zuerst berechnet; die äußere Abfrage nutzt danach das Ergebnis.

Subquery in WHERE - filtern mit Berechnung

Beispiel 1 -- WHERE

-- Kunden ueber dem Durchschnitt

```
SELECT name, total_amount
FROM shop_orders
WHERE total_amount >
      (SELECT AVG(total_amount)
       FROM shop_orders);
```

Beispiel 2 -- IN mit Subquery

-- Produkte die bestellt wurden

```
SELECT product_name
FROM shop_products
WHERE product_id IN
      (SELECT product_id FROM shop_orders);
```

Beispiel 1 erklart:

1. SQL berechnet zuerst den Durchschnitt aller Bestellungen
2. Dann vergleicht WHERE jeden Wert mit diesem Ergebnis
3. Nur Zeilen ueber dem Durchschnitt erscheinen

Beispiel 2 erklart:

IN + Subquery = "ist der Wert in dieser Liste?"
Die Subquery liefert alle product_ids die je bestellt wurden. Nur diese Produkte werden angezeigt.

Merkhilfe:

Subquery in WHERE = dynamischer Filter, der sich selbst berechnet

Subquery in FROM -- Zwischentabelle erzeugen

 Eine Subquery im FROM erzeugt eine temporäre Tabelle -- die du dann wie eine normale Tabelle abfragen kannst.

```
-- Umsatz pro Kunde, nur fuer Kunden mit mehr als 2 Bestellungen

SELECT zusammenfassung.customer_id,
       zusammenfassung.gesamt_umsatz
FROM (
    SELECT customer_id,
           COUNT(*) AS anzahl, SUM(total_amount) AS gesamt_umsatz
    FROM shop_orders GROUP BY customer_id
) AS zusammenfassung
WHERE zusammenfassung.anzahl > 2;
```

1 Subquery (gelb) laeuft zuerst -- berechnet Anzahl und Umsatz pro Kunde

2 Das Ergebnis heisst "zusammenfassung" -- SQL behandelt es wie eine echte Tabelle

3 WHERE filtert -- nur Kunden mit mehr als 2 Bestellungen bleiben im Ergebnis

Primär- & Fremdschlüssel

Primary Key (PK)

- **Eindeutige ID** für jede Zeile
- Darf nicht leer oder doppelt sein
- Jede Tabelle hat genau einen PK

Beispiele:

`customer_id` in `shop_customers`

`order_id` in `shop_orders`

`product_id` in `shop_products`

Foreign Key (FK)

- **Verweis** auf den PK einer anderen Tabelle
- Stellt die Verbindung zwischen Tabellen her
- Der Wert muss im referenzierten PK existieren

Beispiele in `shop_orders`:

`customer_id` zeigt auf `shop_customers.customer_id`

`product_id` zeigt auf `shop_products.product_id`

Wie PK + FK zusammenarbeiten:

In `shop_orders` steht `customer_id = 42`. Wir gehen zu `shop_customers` und suchen die Zeile mit `customer_id = 42` -- das ist Anna. Genau das macht JOIN automatisch für uns!

`ON o.customer_id = c.customer_id` -- das ist der PK-FK-Match

Vom Modell zur Abfrage -- alles zusammen

Frage des Chefs:

"Welche Kunden aus Deutschland haben im letzten Quartal mehr als 500 EUR ausgegeben?"

Was brauchen wir?

- Kundendaten (Land) -- **shop_customers**
- Bestelldaten (Betrag, Datum) -- **shop_orders**
- JOIN ueber **customer_id** (FK)
- WHERE country = 'DE' AND total_amount > 500

```
-- Alles zusammen:  
-- Star Schema + JOIN + WHERE + Subquery
```

```
SELECT c.name,  
       SUM(o.total_amount) AS umsatz  
FROM shop_orders o  
JOIN shop_customers c  
  ON o.customer_id = c.customer_id  
WHERE c.country = 'DE'  
      AND o.total_amount >  
      (SELECT AVG(total_amount)  
       FROM shop_orders)  
GROUP BY c.name  
ORDER BY umsatz DESC;
```

Gelb = Subquery (berechnet Durchschnitt)

Blau = Aeussere Abfrage mit JOIN

11:30–12:10 🍴 Mittagspause





Tag 2 12:10–14:00

Erweiterte SQL-Konzepte & Übung

- Common Table Expressions (CTE)
- Window Functions (z. B. Ranking, laufende Summen)

Was ist ein CTE? - Common Table Expression

CTE = benannte Zwischenabfrage

Ein CTE ist wie eine Subquery in FROM -- aber viel lesbarer.
Du gibst dem Zwischenergebnis einen Namen und verwendest es danach wie eine Tabelle.

Subquery vs. CTE:

- ✗ Subquery: verschachtelt, schwer zu lesen
- ✓ CTE: benannt, wiederverwendbar, klar strukturiert

Syntax:

WITH name AS (SELECT ...) SELECT ... FROM name

-- Umsatz pro Kunde mit CTE

```
WITH kunden_umsatz AS (  
    SELECT customer_id,  
           SUM(total_amount) AS gesamt,  
           COUNT(*) AS anzahl  
    FROM shop_orders  
    GROUP BY customer_id  
)  
  
SELECT c.name, ku.gesamt, ku.anzahl  
FROM kunden_umsatz ku  
JOIN shop_customers c  
    ON ku.customer_id = c.customer_id  
ORDER BY ku.gesamt DESC;
```

Lila = WITH-Block (CTE-Definition)

Gelb = CTE-Inhalt (Zwischenabfrage)

Blau = äussere Abfrage nutzt CTE wie Tabelle

Was sind Window Functions?

Window Functions berechnen Werte über eine Gruppe von Zeilen - ohne die Zeilen zu gruppieren.

GROUP BY reduziert Zeilen

100 Bestellungen GROUP BY customer_id
= 10 Zeilen (eine pro Kunde)

Einzelne Bestellungen verschwinden

OVER() behält alle Zeilen

100 Bestellungen + OVER(PARTITION BY customer_id)
= 100 Zeilen (alle bleiben erhalten!)

Jede Zeile erhält den Gruppenwert zusätzlich

Die wichtigsten Window Functions:

ROW_NUMBER()

Fortlaufende Nummer je Zeile

RANK()

Rang (mit Lücken bei Gleichstand)

SUM() OVER()

Laufende Summe

LAG() / LEAD()

Vorherige / nächste Zeile

Grundstruktur: FUNKTION() OVER (PARTITION BY spalte ORDER BY spalte)

PARTITION BY = Gruppe definieren (optional) | ORDER BY = Reihenfolge innerhalb der Gruppe (für RANK, SUM, LAG)

ROW_NUMBER & RANK -- Kunden ranken

Top-Kunden nach Umsatz ranken

```
SELECT c.name,  
       SUM(o.total_amount) AS umsatz,  
       ROW_NUMBER() OVER (  
         ORDER BY SUM(o.total_amount) DESC  
       ) AS rang,  
       RANK() OVER (  
         ORDER BY SUM(o.total_amount) DESC  
       ) AS rank_mit_luecken  
FROM shop_orders o  
JOIN shop_customers c ON o.customer_id =  
c.customer_id  
GROUP BY c.name;
```

Ergebnis-Beispiel:

name	umsatz	rang	rank_lücken
Anna Mueller	1250	1	1
Ben Schulz	890	2	2
Clara Benz	890	3	2 <-- Lücke!
David Koch	450	4	4

ROW_NUMBER()

Immer 1,2,3,4 -- auch bei
Gleichstand keine Lücken

RANK()

Bei Gleichstand gleicher Rang,
nächster wird übersprungen
(1,2,2,4)

Merkhilfe: OVER() ohne PARTITION BY = über alle Zeilen | OVER(PARTITION BY x) = pro Gruppe

SUM OVER -- laufende Summe berechnen

Laufende Summe = kumulierter Umsatz - wie viel haben wir bis zu diesem Zeitpunkt insgesamt eingenommen?

```
-- Laufende Summe pro Tag
```

```
SELECT
  order_date,
  total_amount,
  SUM(total_amount)
  OVER (
    ORDER BY order_date
    ROWS BETWEEN
      UNBOUNDED PRECEDING
      AND CURRENT ROW
  ) AS laufende_summe
FROM shop_orders
ORDER BY order_date;
```

Ergebnis:

order_date	amount	laufende_summe
2024-01-03	899	899
2024-01-05	29	928
2024-01-08	79	1007
2024-01-10	450	1457 <-- kumuliert!

Grün = ROWS BETWEEN -- definiert das Fenster (alle vorherigen bis aktuelle Zeile)

Vereinfachte Version (oft ausreichend): `SUM(total_amount) OVER (ORDER BY order_date)`

SQLite verwendet standardmässig ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW -- einfach ORDER BY reicht meistens.

Von SQL-Abfrage zum Dashboard-KPI

Aus einer SQL-Abfrage kann ein KPI entstehen, der später in einem Dashboard angezeigt wird.

Business-Frage	SQL-Ergebnis	Dashboard-KPI
Wie viel Umsatz haben wir gemacht?	<code>SUM(total_amount)</code>	Gesamtumsatz
Welche Kunden kaufen am meisten?	<code>GROUP BY customer_id</code>	Top-Kunden
Welche Produkte performen am besten?	<code>GROUP BY product_id</code>	Top-Produkte
Wie entwickelt sich der Umsatz über Zeit?	<code>GROUP BY order_date / month</code>	Umsatztrend
Welche Kundengruppe ist besonders wertvoll?	<code>JOIN + GROUP BY segment</code>	Umsatz nach Segment

Merksatz:

Eine gute SQL-Abfrage ist oft der erste Schritt zu einem guten Dashboard.

14:00–14:20 ☕ Pause





Tag 2 14:20–15:40

Mini-Projekt: Exercise SQL

- Gruppenarbeit mit realem Datensatz



Tag 2 15:40–16:00

Ergebnisse & Diskussion

- Präsentation
- Reflexion
- Q&A

Das nehmt ihr aus Tag 2 mit - 1

Subquery in WHERE

Dynamischer Filter -- zuerst wird das Ergebnis der Subquery berechnet, dann filtert WHERE damit.

```
WHERE x > (SELECT AVG(x) FROM ...)
```

Subquery in FROM

Temporäre Tabelle -- die Subquery liefert ein Zwischenergebnis, das wie eine Tabelle behandelt wird.

```
FROM (SELECT ...) AS name
```

PK & FK -- Tabellen verbinden

PK = eindeutige ID einer Tabelle. FK = Verweis auf einen PK in einer anderen Tabelle. Basis fuer jeden JOIN.

```
ON o.customer_id = c.customer_id
```

Star Schema

Eine Fact Table in der Mitte (Messwerte), mehrere Dimension Tables aussen (Kontext). Verbindung per FK.

```
shop_orders = Fact | customers, products = Dim
```

Das nehmt ihr aus Tag 2 mit - 2

CTE -- WITH ... AS (...)

Benannte Zwischenabfrage. Macht komplexe Abfragen lesbar und wiederverwendbar.

```
WITH name AS (SELECT ...) SELECT ... FROM name
```

OVER() -- Window Functions

Berechnung ueber Gruppen ohne Zeilen zu reduzieren. Alle Zeilen bleiben erhalten.

```
FUNC() OVER (PARTITION BY ... ORDER BY ...)
```

ROW_NUMBER vs RANK

ROW_NUMBER: immer 1,2,3,4. RANK: gleiche Werte bekommen gleichen Rang, naechster wird uebersprungen (1,2,2,4).

SUM() OVER() -- Laufende Summe

Kumulierter Umsatz pro Tag, Monat oder Kunde. Jede Zeile zeigt den Stand bis zu diesem Punkt.

```
SUM(amount) OVER (ORDER BY date)
```

CTE + Window Functions = professionelle SQL-Analysen wie ein echter Data Analyst